Middle East Technical University Department of Computer Engineering

CENG 351

Data Management and File Structures

Fall '2015-2016

In-Class Assignment 4 - B+ Tree

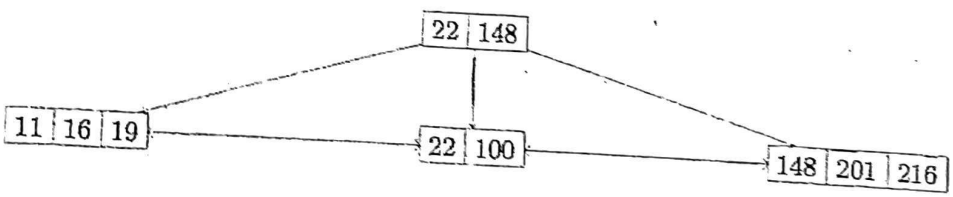
Duration: 75 Minutes

1 Part 1 - B+ Tree Insertion

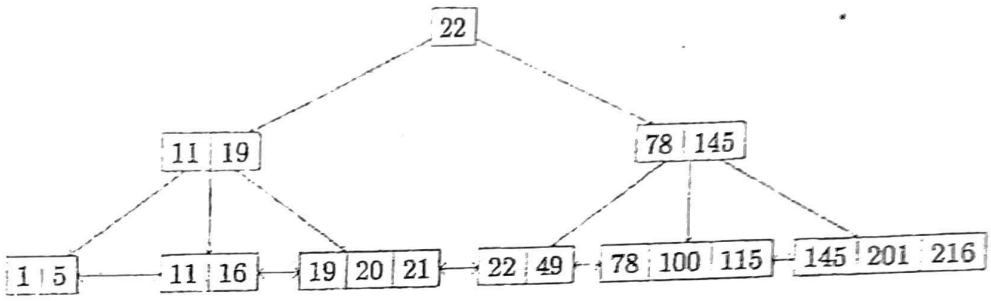
Given the following keys, create a B+ tree of order 2 (each node can hold at most 4 keys) by successive insertions.

[22, 11, 201, 148, 16, 19, 100, 216, 115, 49, 78, 5, 1, 20, 21]

(a) Show the B+ tree after the insertion of 216. *(15 points)*

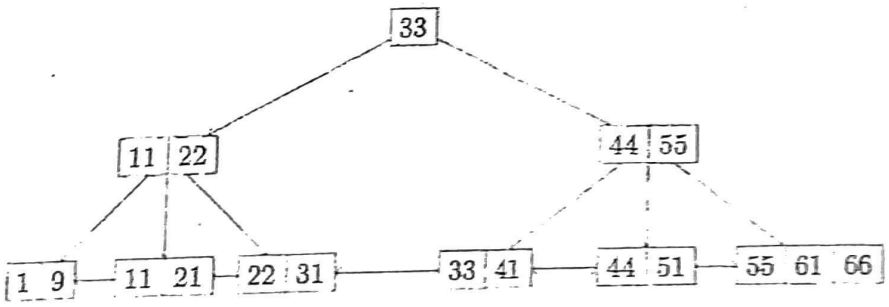


(b) Show the B+ tree after the insertion of 21. *(15 points)*



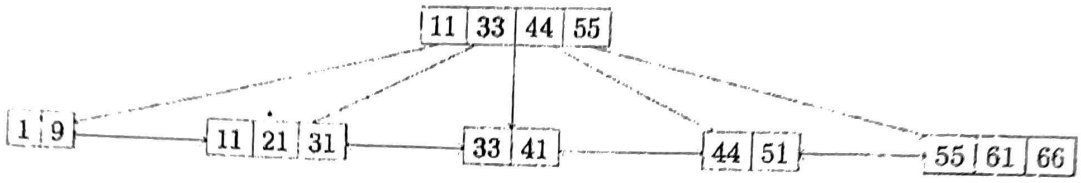
2 Part 2 - B+ Tree Deletion

Given the B+ tree below, apply each of the following delete operations on the given tree above.

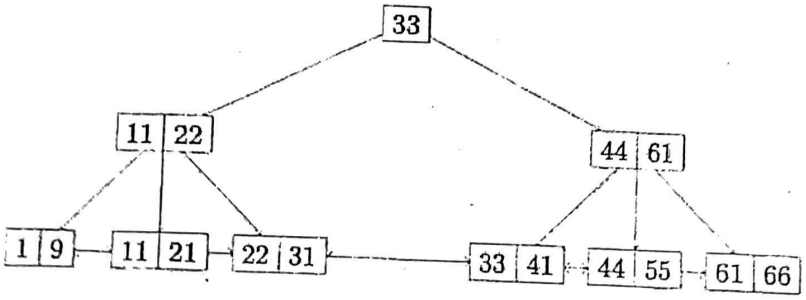


Note: They are all different cases, solve separately.

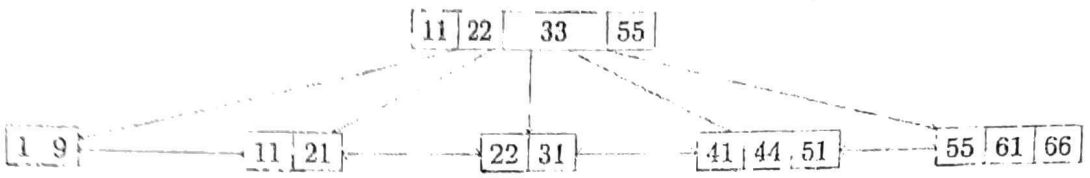
(a) Show the B+ tree (of order 2) after deletion of **22**. *(10 points)*



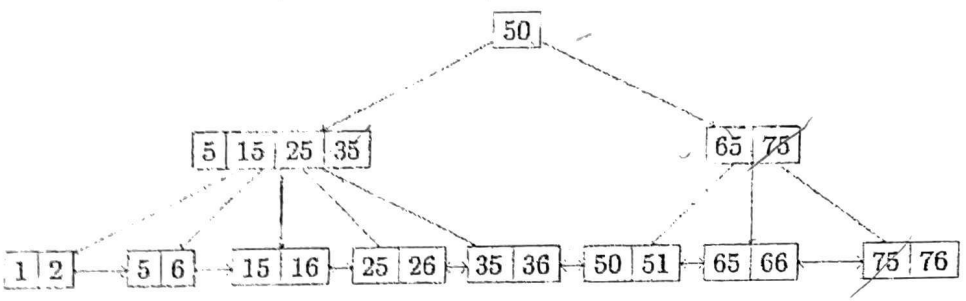
(b) Show the B+ tree (of order 2) after deletion of **51**. *(10 points)*

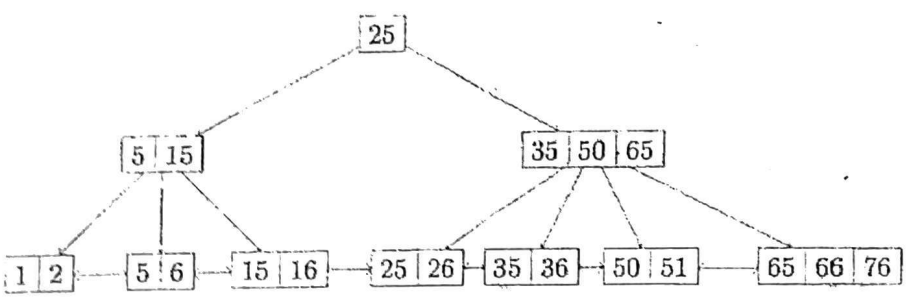


(c) Show the B+ tree (of order 2) after deletion of 33. *(10 points)*



(d) Given the B+ tree below, show the B+ tree (of order 2) after deletion of 75. *(15 points)*





3 Part 3 - Cyber Santa Claus

Merry Christmas! With the advancement of technology, Santa Claus has decided to become a "Cyber" Santa Claus *(I think he saw the Amazon Drone and got a bit jealous)*.

Before investing on drones, he decided to build a database for the gifts, and he needs your help. You are going to create a primary B+ tree index on **giftID** of gift relation attribute.

For the sake of clarity, assume that the gifts are only sorted on **giftID**.

There are **2.500.000** gift records where each record is **400 bytes**. Each **giftID** attribute and address pointer is **12 bytes** and **4 bytes**, respectively.

A bucket can hold **500** records and B+ tree will have a fill factor of **50%**. The data will be kept in a disk *(SSD, Santa's Special Disk, not affiliated with Solid State Disk)* with the following specifications;

- *B (Block Size):* **1600 bytes**
- *btt (Block Transfer Time):* **1 ms**
- *r (Average Rotational Time):* **5 ms**
- *s (Average Seek Time):* **10 ms**

(a) How many bytes are required to form a primary B+ tree for storing the given data? (Note: Do not include the actual data size.) *(5 points)*

$$BucketNumber = \frac{RecordCount}{BucketCapacity * FillFactor} = \frac{2.500.000}{(500 * \frac{1}{2})} = 10.000$$

$$2v = \lceil \frac{1600}{12+4} \rceil = 100$$

$$AverageFanOut = 2v * FillFactor = 100 * \frac{50}{100} = 50$$

For 10.000 buckets, there should be

$$\lceil \frac{10000}{50} \rceil = 200$$

keys over buckets. For 200 keys, there should be

$$\lceil \frac{200}{50} \rceil = 4$$

keys in upper level. For 4 keys, there should be

$$\lceil \frac{4}{50} \rceil = 1$$

key in upper level.

$$NumberOfKeys = 200 + 4 + 1 = 205$$

$$\textbf{RequiredBytes} = NumberOfKeys * KeyBlockSize = 205 * 1600 = \textbf{328.000 bytes}$$

(b) How long will it take to find and read an arbitrary record in an arbitrary bucket? (Note: Assume that there are no indexes or records in the memory. You just know the address of the root index block.) *(10 points)*

** Since there are 3 layers;

$$Result = 3 * (CostOfReadingALayer) + (CostOfReadingABucket)$$

$$Result = 3 * (s + r + btt) + (s + r + dtt)$$

$$dtt = NumberOfBlockForABucket = \lceil \frac{500 * 400}{1600} \rceil = 125\ blocks$$

$$\textbf{Result} = 3 * (10 + 5 + 1) + (10 + 5 + 125 * btt) = 48 + 140 = 188\ ms$$

(c) How long will it take to read all the records in a sorted manner? (Note: Assume that there are no indexes or records in the memory. You just know the address of the root index block. Also, the buckets are linked to each other in a sorted manner.) *(10 points)*

$$Result = 3 * (CostOfReadingALayer) + (BucketNumber * CostOfReadingABucket)$$

$$\textbf{Result} = 48 + 10.000 * 140 = \mathbf{1.400.048}\ ms$$